

SCIENTIFIC COMPUTING



Peter Staab
Fitchburg State University

Book: Scientific Computing (Staab)

This text is disseminated via the Open Education Resource (OER) LibreTexts Project (<https://LibreTexts.org>) and like the thousands of other texts available within this powerful platform, it is freely available for reading, printing, and "consuming."

The LibreTexts mission is to bring together students, faculty, and scholars in a collaborative effort to provide an accessible, and comprehensive platform that empowers our community to develop, curate, adapt, and adopt openly licensed resources and technologies; through these efforts we can reduce the financial burden born from traditional educational resource costs, ensuring education is more accessible for students and communities worldwide.

Most, but not all, pages in the library have licenses that may allow individuals to make changes, save, and print this book. Carefully consult the applicable license(s) before pursuing such effects. Instructors can adopt existing LibreTexts texts or Remix them to quickly build course-specific resources to meet the needs of their students. Unlike traditional textbooks, LibreTexts' web based origins allow powerful integration of advanced features and new technologies to support learning.



LibreTexts is the adaptable, user-friendly non-profit open education resource platform that educators trust for creating, customizing, and sharing accessible, interactive textbooks, adaptive homework, and ancillary materials. We collaborate with individuals and organizations to champion open education initiatives, support institutional publishing programs, drive curriculum development projects, and more.

The LibreTexts libraries are Powered by [NICE CXone Expert](#) and was supported by the Department of Education Open Textbook Pilot Project, the California Education Learning Lab, the UC Davis Office of the Provost, the UC Davis Library, the California State University Affordable Learning Solutions Program, and Merlot. This material is based upon work supported by the National Science Foundation under Grant No. 1246120, 1525057, and 1413739.

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation nor the US Department of Education.

Have questions or comments? For information about adoptions or adaptations contact info@LibreTexts.org or visit our main website at <https://LibreTexts.org>.

This text was compiled on 06/08/2025

TABLE OF CONTENTS

Licensing

Preface

1: Introduction to Scientific Computing

- 1.1: Examples of Scientific Computing
- 1.2: Modeling
- 1.3: Computing
- 1.4: Ideas needed to do Effective Scientific Computing
- 1.5: Examples of Scientific Computing
- 1.6: Writing Code for Scientific Computing
- 1.7: Let's get started

2: Storing number and strings and using Julia syntax

- 2.1: Numbers
- 2.2 Assignment Statement and Variables
- 2.3: Strings
- 2.4: Expressions and Operator Precedence

3: Introduction to Data Types

- 3.3: Extending integers, the BigInt type
- 3.1: Integers
- 3.2: Floating Point Numbers
- 3.4: Rational and Complex Numbers

4: Introduction to Functions

- 4.1: Simple Examples of Functions
- 4.2: Input and Output of Functions
- 4.3: Multiple Dispatch
- 4.4: Variable Number of arguments
- 4.5: Factorial Function
- 4.6: Variable Scope

5: Boolean Statements, Loops and Branching

- 5.1: Boolean values and if statements
- 5.2: If Statements
- 5.3: While Loops

6: Arrays

- 5.1: Introduction to Linear Algebra
- 5.2: Advanced Topics in Linear Algebra

7: Functional Programming and an Introduction to Writing fast code

- 6.1: Numerical Integration
- 6.2: Differential Equations
- 6.3: Optimization
- 6.4: Complex Numbers and Analysis

8: Number theory and Algorithm Development

- 7.1: Calculating pi
- 7.2: Newton's Method in the Complex Plane
- 7.3: Installing and Getting Started with Julia
- 7.4: Loading Packages
- 7.5: Binary Representation of Numbers
- 7.6: References

9: Algorithm Analysis

10: Solving Quadratics and Rootfinding

- 10.1: Absolute and Relative Errors
- 10.2: Rounding Errors and the Quadratic Formula
- 10.3: Re-examining the Quadratic Formula
- 10.4: Newton's Method

11: Plotting Data and Functions

- 11.1: The Plots Package
- 11.2: Presenting Results with Visualization, an overview
- 11.3: Other Function Plots
- 11.4: Plotting Data

[Index](#)

[Glossary](#)

[Glossary](#)

[Detailed Licensing](#)

Licensing

A detailed breakdown of this resource's licensing can be found in [Back Matter/Detailed Licensing](#).

Preface

Solving problems is the essence of scientific computing. Many problems arise naturally from the sciences or mathematics and often to solve difficult problem relies on some computing resources. In a nutshell, this text is the blend of science, mathematics and computer science needed to solve problems.

The intended audience for this text one who is interested in writing computer code that will solve a problem that you have. There are a few basics that you should have with this text:

- Some background in writing computer programs. Your base language does not matter, but you should be familiar in writing short programs.
- Some calculus. Most of the examples here do not need Calculus, but there are topics throughout the book, that at least differential calculus is helpful.

As many textbooks, this started as a set of notes for my Scientific Computing class at Fitchburg State University. The students in the class are usually either Computer Science or Mathematics major and generally have an interest in computing. As I steered these pages from a set of notes to a textbook, I still keep those students in mind. However, I realize that an undergraduate text is often picked up by graduate students, those in industry and other faculty to build some knowledge base that one never had. I keep these readers in mind as well.

I also want to distinguish this text from that of a Numerical Analysis or Numerical Methods text. A Numerical Methods text often covers many basic algorithms in mathematics such as numerical differentiation, numerical integration, rootfinding, linear algebra, etc. and how to implement such algorithms; this can be done with pseudocode or with a particular language. Numerical Analysis often covers the same algorithms in Numerical Methods with an emphasis on the analysis of the algorithms. This text does not broadly cover all the topics in Numerical Methods, nor does it do any analysis. There are many good books listed in the references. Instead, this text covers how to solve particular problems and often the algorithms have been coded in an existing library and we will leverage those here.

Why use Julia or any particular language?

The subtitle of the text says that solving problems in Scientific Computing does not happen in a vacuum or necessarily in the land of pseudocode, so I have specifically chosen a language to explain the concepts in the text much like I do in my class and the computing language that will be used will be Julia.

Before going through the technical reasons for choosing Julia, the following personal story is interesting. I proposed teaching Scientific Computing at Fitchburg State University and the class would run in the Fall of 2014. The preceding spring, I knew that I had to choose some language for the class. There are many languages which would have been fine to use, including Maple, which our majors were learning in a required class, and that would have been an easy choice. However, one of blogs that I read regularly, , had an article by Lee Philips entitled ``Scientific computing's future: Can any coding language top a 1950s behemoth?" The essence of the article is that in scientific computing, Fortran is king, due to two reasons 1) it is fast mainly because it was designed to be easily translated to machine language, which eventually is what runs on a computer and 2) there are many important mathematical libraries written in Fortran have been time-tested and virtually bug-free.

However, nobody wants to code in Fortran. It is missing many features of modern languages and requires compilation. In selecting Julia, as stated on the main webpage, ``Julia was designed from the beginning for high performance." As problems in Scientific Computing increase in size and complexity, it is an imperative that code runs fast and Julia typically does. It is one of the fastest high-level computing languages in benchmarks (REF) and compared to other scientific computing languages, it is the fastest. At the time the article was written, Julia was in version 0.2 and was quite raw, but was fascinating to consider and in hind sight, I'm glad I have done so.

Another important aspect of Julia is that it is a scripting language like python. This means that you have an environment setup, you can just start entering julia commands and go. There is no compilation involved (like that in C or Fortran). Although compilation often finds errors, it is slow to do prototyping.

How to read this text

How might one read this text? Of course, the short answer is anyway you want. I hope you can pick it up and read what is relevant. There's a few items about the layout as well as some standard conventions.

Take a look at the table of contents. The text is broken up into a few parts. Part I consists of some basics of both the julia language as well as scientific computing. The remaining parts of the text are separate sections (Simulation, Data Analysis, Linear Algebra) and some advanced features of julia. Once Part I is read, the remaining parts are independent, however there are some ideas in Part III that are very helpful later in the text. The following diagram shows the chapter dependencies:

In addition, because there are code snippets throughout the text, you should run the code as you read the text. Chapter 1 is the beginning to getting you started to run julia, but it is recommended to read Appendix A for more details. All julia code is written in and is either embedded in the text, like or written in a separate block like

```
A = [1,2,3]
```

Any blocks of code should be able to be copied and pasted directly into the julia REPL or a jupyter notebook.

Also, output from julia is also in a fixed-width font but generally in a block like this:

```
"This is a string of output"
```

Here's some of the coding style that is used throughout the text as well. This helps with readability:

- functions are written in camelCase, like `findRealRoot` .
- if a function returns a boolean (true or false) it should start with is or has, like `isFullHouse` from Chapter 15
- variables are written in `snake_case` like `num_trials`

When writing your code, you don't have to follow this convention, but I have tried to stay consistent in the text. Generally this convention will help with readability in that you know directly what kind of variable or function something is.

Code blocks

All of the code blocks in the text are written and run as julia code (see the next section below). Therefore, you should be able to copy any of the blocks out and paste them directly. The only issue might be for long lines. For example if you have

If you try to copy and paste the above line, it will give an error until you take out the arrow characters indicating a line wrap and putting it back on a single line. After this, it should run.

Creating this Text

I think it's interesting how this text was created. JuliaCon (which will be discussed later) is an annual conference on julia. In 2019, Tim Wheeler gave the talk entitled . I based a lot of this textbook on their work. Here's some specific details about that:

There are a couple of reasons for writing it this way and they are related. First, it eliminates errors that often occur by copying and pasting from somewhere where code is run into the document. Secondly,

CHAPTER OVERVIEW

1: Introduction to Scientific Computing

This page is a draft and is under active development.

So you picked up (or probably actually clicked on) this text and maybe you wonder what Scientific Computing actually is. This will give a brief overview of the field and try to explain with some actual examples. In short, scientific computing is a field of study that solves problems from the sciences and mathematics that are generally too difficult to solve using standard techniques and need to resort to writing some computer code to get an answer.

[1.1: Examples of Scientific Computing](#)

[1.2: Modeling](#)

[1.3: Computing](#)

[1.4: Ideas needed to do Effective Scientific Computing](#)

[1.5: Examples of Scientific Computing](#)

[1.6: Writing Code for Scientific Computing](#)

[1.7: Let's get started](#)

This page titled [1: Introduction to Scientific Computing](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

1.1: Examples of Scientific Computing

The classic example of scientific computing is that of weather prediction. Atmospheric scientists and meteorologists use weather models (mathematical equations) that are run on fairly powerful computers. The results are generally a large amount of data (temperature, pressure, humidity, etc.) that often is plotted for various regions of the U.S. or other countries. The resulting data gives the standard forecasts that many of us look at on smart phone apps or on TV or other places.

Data analysis is also ubiquitous and one can think about this falling in the realm of scientific computing. Although this varies, an autonomous vehicle can generate 25 Gb of data per day.. Although there is plenty of scientific computing without the data generation in self-driving cars, this is an example of needing to handle and analyze large amount of data quite quickly.

This year, disease modeling came into focus with the COVID-19 pandemic. Plenty of research posted updates on predicted cases and deaths in the U.S. and throughout the world. Most of these model involve using existing data to fit mathematical functions and then use the resulting functions to predict the future. In addition, due to the diversity of conditions in different regions of the country and world, many localized models are developed at once with some predicted interaction as people travel from region to region.

This page titled [1.1: Examples of Scientific Computing](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

1.2: Modeling

In nearly every example of scientific computing, a mathematical model is used. In short, a model is an equation, set of equations or some algorithm that is assumed as a mathematical understanding of the underlying problem. Mathematical models range vastly from field to field and generally the details are only taught in the individual fields, so we're not covering too much about models in this course, but here's a few examples.

- In weather prediction, if the underlying physics is assumed, there is a large set of partial differential equations called the Navier-Stokes equations. Some approximation of these is used often with some statistical modeling for weather models.
 - In the autonomous car example above, modeling of the car dynamics often consists of the physics of moving objects which link position, velocity, acceleration and force. There is also a lot of artificial intelligence (AI) for object detection.
 - In the contagious disease models, there is usually some use of differential equations (the SIR model is a simple version) and some probability modelling. We will discuss both of these in this text.
-

This page titled [1.2: Modeling](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

1.3: Computing

The computing side of Scientific Computing is generally very important as well. As mentioned above, most of the problems studied get much too large or complicated to solve analytically (generally as a mathematical problem) and some sort of approximation is needed.

In many cases, there are existing algorithms that are developed to solve some underlying mathematical problem and the main focus is on setting up the problem in the right way. For example, if there is a differential equation to solve, a package can be loaded to use a particular differential equation solver and then the results need to be analyzed.

But in general, those that succeed in Scientific Computation are good at the coding/computation side of things. It takes such skills to manage problems common in this field.

I want to emphasize that Scientific computation, however, is not just computer programming. It takes a blend of skills to succeed at this. One needs a fairly deep knowledge of the field they are working in to understand how solving the problem using some algorithm is generating the correct answer.

This page titled [1.3: Computing](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

1.4: Ideas needed to do Effective Scientific Computing

Scientific Computing is generally used to solve problems in Mathematics and various Science fields. There are a number of important things that you need to know to solve problems effectively.

- Find/develop code that runs quickly. Everyone wants the answer as soon as possible. However, if you write some code that doesn't take long to develop and returns the correct answer in a few seconds, it probably doesn't matter how efficient the algorithm is. However, if it takes a few hours to run, you probably want to investigate your solution algorithm.
 - Find/develop code that uses an appropriate amount of memory. Another important aspect is that of memory consumption. It's not hard to find datasets these days that are 1TB or more in size, however few desktop/laptop machines have more than 16 or 32Gb of RAM, so you can't load the entire file into memory. Such a dataset would need to be processed in chunks.
 - Make sure you have known solutions/unit tests How do you know that your solution is correct? It is important to have relatively simple cases that you can test your code on before solving more complex things.
-

This page titled [1.4: Ideas needed to do Effective Scientific Computing](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

1.5: Examples of Scientific Computing

- Consider a list of 5 cities that you need to visit and return to where you started. If you know the distances between each pair of cities, what is the most efficient path (in terms of distance or time) that gets you to each city and return.
- Poker is a card game in which certain sets of cards (hands) beat other hands. Determine the probability (or chance) of getting any particular hand in Poker.
- Given a dataset of Major League Baseball scores from 2000-2017, find the game with the largest score, most innings played, biggest winning margin, etc. We will look at these and other problems in this course.

We will look at these and other problems in this course.

This page titled [1.5: Examples of Scientific Computing](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

1.6: Writing Code for Scientific Computing

Look back at the list of requirements for Scientific Computing projects. A general rule of thumb is to use a computing language that you know will work well for those tasks. Typically, most students have learned one or two languages and aren't quite sure which to use. In most cases it doesn't matter what to use, but for complex problems it will matter.

One of the best languages for scientific computing has been Matlab over the past 3 or 4 decades. It is used extensively in engineering firms throughout the world (and the headquarters is in nearby Natick, MA). We won't be using Matlab here though. One of the nice things about taking a class is exploring new languages. Here we will use [Julia](#), which is a very new language that a lot of people in scientific computing are excited about. We will give examples using Julia, but the ideas here should be applicable to other languages.

Julia was designed as a scientific computing language, but in short is a modern language. There are number of aspects that makes julia a good language for this. Julia is

- *a scripting language with dynamic types*. This means you can get started right away—there's no need to learn about compilation—and you can prototype things quickly.
- *a language with just-in-time compilation*. Most of the time scripting languages are slow, however with modern under-the-hood tools, languages can be compiled on the fly to create very fast runtimes. There is a [webpage on benchmarks](#) comparing julia to other standard languages.
- *open source*. Although often open-source implies free but not high-quality, the free part holds, but more important is that anyone can contribute to the code. The Julia community is committed to creating a high-quality piece of software and many discussion revolve around writing code that will improve the speed or other aspects. This is very opaque and unclear if it happens with commercial software.
- *easy to use*. The syntax of julia is similar to that of python, a very popular language, and is often intuitive.
- *a joy to use*. Additionally, the structure of the language makes it easy to start with, but as projects get more complex, it can be written in a simple way.

This page titled [1.6: Writing Code for Scientific Computing](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

1.7: Let's get started

You can start by just reading the book, but to get the most out of this, make sure that you have a computer with access to julia. If you already have this, great. If not, see Appendix which will show you a few way to get the code up and running.

This page titled [1.7: Let's get started](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

CHAPTER OVERVIEW

2: Storing number and strings and using Julia syntax

Solving Scientific Computing problems ultimately boils down to manipulating data and at the most basic is that of strings and numbers. We begin with understanding these data types and how to store values in them.

We also show some of julia syntax, which looks like other languages (like python). Hopefully you have some basic knowledge of computing, but no assumption of any particular language is necessary.

[2.1: Numbers](#)

[2.2 Assignment Statement and Variables](#)

[2.3: Strings](#)

[2.4: Expressions and Operator Precedence](#)

This page titled [2: Storing number and strings and using Julia syntax](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

2.1: Numbers

Solving Scientific Computing problems ultimately boils down to manipulating data and at the most basic is that of strings and numbers. We begin with understanding these data types and how to store values in them.

We also show some of julia syntax, which looks like other languages (like python). Hopefully you have some basic knowledge of computing, but no assumption of any particular language is necessary.

Numbers

Not surprisingly, the most important data type in scientific computing—at least at the most atomic level is numbers. Numbers in computation mimic those in mathematics with some important differences. Julia like most computing languages have two main number types, integers and floating points. Julia's integers and much like mathematical integers in that they store numbers like $\backslash(0, 10, -400\backslash)$. Floating-point numbers generally are approximations to real or decimal numbers and more details will be covered in Chapter ??.

Julia also has a rational data type for numbers commonly thought of as fractions like $-\frac{2}{3}$ or $\frac{22}{7}$. Recall that typically i represents $\sqrt{-1}$, the base imaginary number. Complex numbers like $2 + 3i$ are also native to julia. We will explore both rational and complex numbers in greater depth in Chapter ??.

Assignment Statements and Variables

Anything can be stored as a variable using the single equal sign like



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=6
```

This is an assignment operator, which creates the number 6 and stores it under the name `x`.

And now that the variable is stored, we can use it in calculations. For example



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x+3
```

```
9
```

Variables in Julia, much like other languages are primarily sequences of alphanumeric characters as well as an underscore. Primarily, a variable needs to start with an alphabetic character or after the first character can contain numbers.

Julia also allows many unicode symbols in variable names, however not everything. For example, all of the Greek letters are allowed, so $\alpha = 45$ is valid.

To get a Greek letter in Jupyter or the REPL, type `\alpha`, hit the TAB key and it will be turned into an α .

Storing Variables in a Virtual Whiteboard

The details of storing variables in computer hardware isn't necessary, however, thinking of storing as writing variables and values on a whiteboard is a helpful paradigm. Imagine a whiteboard with a column of variable names and a column of values. For example, if we have



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=6  
y=-1  
z=8.5
```

```
8.5
```

then you can think of the whiteboard looking like:

variable	value
x	6
y	-1
z	8.5

If we evaluate any expression containing any of these variables, then the value looked up substituted into the expression. For example,



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
2*y
```

```
-2
```

looks up the value of y (which is -1) and substitutes that value in and multiplies the result by 2. As you can see the result is -2.
If we change one of the values, like



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
y = y+5
```

this means that the right hand side is evaluated by looking up the value of y and the result is 4. Then the 4 is placed into the whiteboard, which will now look like:

variable	value
x	6
y	4
z	8.5

If you are thinking of how a piece of code works, often you will need to get to the point of writing down a version of the whiteboard.

Strings

In many Scientific Computing fields, such as Data Science, strings arise often and it is important to understand some of the basics of them. In Julia, a string is a sequence of characters surrounded by `" "` (double quotes). For example:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
str = "This is a string"
```

and if you enter



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
typeof(str)
```

which shows that it is a `String`. Section 1.2 will go into more details about data types in Julia. The individual parts of the string are called characters, which are of type `Char` and are by default Unicode Characters (which will we see are super helpful). A few other helpful things about strings are

- The length of a string is found using the length command. `length(str)` returns 16.
- To access the first element of the string, type `first(str)`, the last is found by `last(str)` and the 3rd character for example is `str[3]`. In julia, string indexing starts at 1
- To turn other data types into string, use the `string` function. For example



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
string(3.0)
```

```
"3.0"
```

and again note that the double quotes indicate that this is a string.

String Operations

A common string operations is concatenation, or the merging of two strings. Consider



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
str1 = "The tide is high "  
str2 = "and I'm having fun."
```

We can concatenate in two ways, with the `*` operator symbol or the `string` function. Both



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
str1 * str2  
string(str1,str2)
```

```
"The tide is high and I'm having fun."
```

returns the same string concatenation. I find the second option clearer in that `*` is an odd choice for string concatenation. Many languages including java and ecmaScript (javascript) use `+` instead for string concatenation, which I think intuitively makes more

sense.

Another cute operation for strings is the caret operation. This could be helpful, however a not-so-helpful example is



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
"Hip, hip, hooray! "^3
```

```
"Hip, hip, hooray! Hip, hip, hooray! Hip, hip, hooray! "
```

concatenates the string three times. Other important functions related to strings can be found at [JULIA DOCUMENTATION ON STRINGS](#)

String Interpolation

Mixing strings and other variables together is often needed and we can use the methods of concatenation as above. However, there is a better way. Let's use the value of `x` that we stored above and insert it at the end of `"The value of x is "`. The following will do this



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
result = "The value of x is $x"
```

There is actually another way to do string concatenation if you use this method. Using the variables stored in `str1` and `str2`, then





Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
"$str1$str2"
```

will also concatenate the strings.

We can also go beyond just looking up strings in that we can execute code as well. Let's say that we have the variable



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
n=6
```

If we want a string with its square, we can enter:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
"The square of $n is $(n^2)."
```

```
"The square of 6 is 36."
```


where you should notice that the code `n^2` is executed since that code is within the `$()`.

Exercise

Use the volume of a sphere $\frac{4}{3}\pi r^3$, to calculate the volume and produce a string. For example, store `r=3` and then return the result

```
"The volume of a sphere of radius 3 is 113.0973355292"
```

where 3 and the volume is generated with string interpolation.



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

Expressions

An expression is a combination of variables, data elements (like numbers and strings), operations (like `+` or `*`) and functions (like `.`). We've seen a number of expressions throughout this chapter so far like



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=6
x+3
str1 * str2
length(str)
```

In short, writing things in julia will consist of writing expressions (and slightly more complicated structures).

Operator Precedence

When we type out an expression like `11+2*(4+3)^3`, it is important to understand the order in which operators are performed. For mathematics, the PEMDAS pnemonic is helpful to remember in that the order is:

- *Parentheses*: The expression inside the () are done first. For the example above, the `4+3` is the first operation done.
- *Exponentials*: The `^` is done next. Raise the 7 from above to the power of 3 resulting in `343`.
- *Multiplication and Division*: In this example, the `2*(343)` is done next
- *Addition and Subtraction*: Lastly add 11 to the result and the result is `697`.

In any computing language, there are other operators as well and there is order to that precedence, so we will see that there are other things to think about. For example, the assignment operator, has the lowest precedence. That is when assigning something to a variable, all calculations are done on the right side of the `=` before the assignment.

Details on all this can be found on the [JULIA DOCUMENTATION ON OPERATOR PRECEDENCE](#)

Comments

A comment in computer code is sequences of characters which are ignored. The purpose of a comment is to alert a human on what is going on. You may have been told to write comments so that someone else who reads your code understands what you are doing. However, I have found that the person mostly like to read your code is you at a later date. You should add comments for yourself.

In julia, a comment is anything to the right of a , pound sign or hash tag. For example:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
## This calculates the area of a circle
r=3
pi*r^2 # this is the actual formula for the area
```

Both lines 1 and 3 have comments. On line 1, the entire line is ignore since the line starts with `#`. On line 3, everything after the `2` (the power) is ignored. Also, notice that there are two hash tags on line 1 and 1 on line 3. This is simply different style. Since anything after a single `#` is a comment, everything after the first one is ignored.

2.1: Numbers is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by LibreTexts.

2.2 Assignment Statement and Variables

Assignments

Anything can be stored as a variable using the single equal sign like



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=6
```

This is an assignment operator, which creates the number 6 and stores it under the name `x`.

And now that the variable is stored, we can use it in calculations. For example



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x+3
```

```
9
```

Variables in Julia, much like other languages are primarily sequences of alphanumeric characters as well as an underscore. Primarily, a variable needs to start with an alphabetic character or after the first character can contain numbers.

Julia also allows many unicode symbols in variable names, however not everything. For example, all of the greek letters are allowed, so $\alpha = 45$ is valid.

To get a greek letter in Jupyter or the REPL, type `\alpha`, hit the TAB key and it will be turned into an α .

Storing Variables in a Virtual Whiteboard

The details of storing variables in computer hardware isn't necessary, however, thinking of storing as writing variables and values on a whiteboard is a helpful paradigm. Imagine a whiteboard with a column of variable names and a column of values. For example, if we have



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=6
y=-1
z=8.5
```

```
8.5
```

then you can think of the whiteboard looking like:

variable	value
x	6
y	-1
z	8.5

If we evaluate any expression containing any of these variables, then the value looked up substituted into the expression. For example,



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
2*y
```

-2

looks up the value of y (which is -1) and substitutes that value in and multiplies the result by 2. As you can see the result is -2. If we change one of the values, like



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

$y = y+5$

this means that the right hand side is evaluated by looking up the value of y and the result is 4. Then the 4 is placed into the whiteboard, which will now look like:

variable	value
x	6
y	4
z	8.5

If you are thinking of how a piece of code works, often you will need to get to the point of writing down a version of the whiteboard.

[2.2 Assignment Statement and Variables](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by LibreTexts.

2.3: Strings

Strings in Julia

In many Scientific Computing fields, such as Data Science, strings arise often and it is important to understand some of the basics of them. In Julia, a string is a sequence of characters surrounded by `" "` (double quotes). For example:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
str = "This is a string"
```

and if you enter



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
typeof(str)
```

which shows that it is a `String`. Section 1.2 will go into more details about data types in Julia. The individual parts of the string are called characters, which are of type `Char` and are by default Unicode Characters (which will we see are super helpful). A few other helpful things about strings are

- The length of a string is found using the length command. `length(str)` returns 16.
- To access the first element of the string, type `first(str)`, the last is found by `last(str)` and the 3rd character for example is `str[3]`. In julia, string indexing starts at 1
- To turn other data types into string, use the `string` function. For example



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
string(3.0)
```

```
"3.0"
```

and again note that the double quotes indicate that this is a string.

String Operations

A common string operations is concatenation, or the merging of two strings. Consider



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
str1 = "The tide is high "  
str2 = "and I'm having fun."
```

We can concatenate in two ways, with the `*` operator symbol or the `string` function. Both



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
str1 * str2
string(str1,str2)
```

```
"The tide is high and I'm having fun."
```

returns the same string concatenation. I find the second option clearer in that `*` is an odd choice for string concatenation. Many languages including java and ecmaScript (javascript) use `+` instead for string concatenation, which I think intuitively makes more sense.

Another cute operation for strings is the caret operation. This could be helpful, however a not-so-helpful example is



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
"Hip, hip, hooray! "^3
```

```
"Hip, hip, hooray! Hip, hip, hooray! Hip, hip, hooray! "
```

concatenates the string three times. Other important functions related to strings can be found at [JULIA DOCUMENTATION ON STRINGS](#)

String Interpolation

Mixing strings and other variables together is often needed and we can use the methods of concatenation as above. However, there is a better way. Let's use the value of `x` that we stored above and insert it at the end of `"The value of x is "`. The following will do this



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
result = "The value of x is $x"
```


There is actually another way to do string concatenation if you use this method. Using the variables stored in `str1` and `str2`, then



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
"$str1$str2"
```

will also concatenate the strings.

We can also go beyond just looking up strings in that we can execute code as well. Let's say that we have the variable



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
n=6
```

If we want a string with its square, we can enter:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
"The square of $n is $(n^2)."
```

```
"The square of 6 is 36."
```

where you should notice that the code `n^2` is executed since that code is within the `$( )`.

Exercise

Use the volume of a sphere $\frac{4}{3}\pi r^3$, to calculate the volume and produce a string. For example, store `r=3` and then return the result

```
"The volume of a sphere of radius 3 is 113.0973355292"
```

where 3 and the volume is generated with string interpolation.



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

2.3: Strings is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by LibreTexts.

2.4: Expressions and Operator Precedence

Expressions

An expression is a combination of variables, data elements (like numbers and strings), operations (like + or *) and functions (like). We've seen a number of expressions throughout this chapter so far like



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=6
x+3
str1 * str2
length(str)
```

In short, writing things in julia will consist of writing expressions (and slightly more complicated structures).

Operator Precedence

When we type out an expression like $11+2*(4+3)^3$, it is important to understand the order in which operators are performed. For mathematics, the PEMDAS mnemonic is helpful to remember in that the order is:

- *Parentheses*: The expression inside the () are done first. For the example above, the $4+3$ is the first operation done.
- *Exponentials*: The $^$ is done next. Raise the 7 from above to the power of 3 resulting in 343 .
- *Multiplication and Division*: In this example, the $2*(343)$ is done next
- *Addition and Subtraction*: Lastly add 11 to the result and the result is 697 .

In any computing language, there are other operators as well and there is order to that precedence, so we will see that there are other things to think about. For example, the assignment operator, has the lowest precedence. That is when assigning something to a variable, all calculations are done on the right side of the = before the assignment.

Details on all this can be found on the [JULIA DOCUMENTATION ON OPERATOR PRECEDENCE](#)

Comments

A comment in computer code is sequences of characters which are ignored. The purpose of a comment is to alert a human on what is going on. You may have been told to write comments so that someone else who reads your code understands what you are doing. However, I have found that the person mostly like to read your code is you at a later date. You should add comments for yourself.

In julia, a comment is anything to the right of a , pound sign or hash tag. For example:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
## This calculates the area of a circle  
r=3  
pi*r^2 # this is the actual formula for the area
```

Both lines 1 and 3 have comments. On line 1, the entire line is ignored since the line starts with `#`. On line 3, everything after the 2 (the power) is ignored. Also, notice that there are two hash tags on line 1 and 1 on line 3. This is simply different style. Since anything after a single hash is a comment, everything after the first one is ignored.

[2.4: Expressions and Operator Precedence](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by LibreTexts.

CHAPTER OVERVIEW

3: Introduction to Data Types

[3.3: Extending integers, the BigInt type](#)

[3.1: Integers](#)

[3.2: Floating Point Numbers](#)

[3.4: Rational and Complex Numbers](#)

This page titled [3: Introduction to Data Types](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

The result is 0, which should be surprising, however note that because of order of operations, first 10^{100} is calculated in standard `Int64` and then turned into a `BigInt`. Again, for details on what happens here, look at Appendix C, in short this continually multiplies the number ten, 100 times which results in overflow and as such results in 0.

It is recommended only to use a `BigInt` if needed. As we will see in chapter 8, operations with them are significantly slower than `Int64` or even `Int128`. Under few cases do you need to do this. For example, in that later chapter with prime numbers.

The BigFloat Datatype

Let's revisit an example from section XXXX and sum $1/9$ nine times. Recall that



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
1/9+1/9+1/9+1/9+1/9+1/9+1/9+1/9+1/9
```

```
1.00000000000000002
```

which shows the limitation of the floating-point type. The `BigFloat` datatype is more flexible. However, note that



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
a=big(1/9)
```

```
0.1111111111111111104943205418749130330979824066162109375
```

which seems to have an accuracy of only 17 digits, which is typical for a 64-bit floating point, so it doesn't appear to have improved anything. This, like above, is a case of being careful in constructing a `BigFloat`. What happens with `big(1/9)` ?



which looks more like an expected result. To determine the number of digits of accuracy, you can count (painfully) or try



which is about 5 times the accuracy of `Float64`. Note: looking again at order of operations, the command `length(string(a))` first takes the number `a` and returns it as a `String`. Then working inside to outside, find the length of the string.

The `precision` function used on a variable of type `BigFloat` will tell the precision of the variable. Thus



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
precision(a)
```

```
256
```

which is the number of bits of precision and it has 4 times the binary precision of `Float64` but about 5 times the decimal precision.

As noted at the beginning of this section, though, if we want to compute π to 1 million decimal digits, what we've seen so far only has about 80 digits of accuracy. As a bit of an enticement, we will do this in Chapter XXX.

The `BigFloat` type is quite flexible. The above example used it in its default precision. We can change this with the `setprecision` function. For example:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
setprecision(2^10)
```

```
1024
```

showing that now `BigFloats` will be stored in this many bits. Such a number will use 16 times the number of binary digits as a `Float64`. Entering



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
a2=big(1)/big(9)
```

[illegible]

a number that, at a glance, looks to store about 4 times the number of digits as the default `BigDecimal` size. To determine the total number of decimal digits, if we enter



```
length(string(big(a2)))
```

311

as the number of decimal digits. This is about 20 times the precision of `Float64` .

This page titled [3.3: Extending integers, the BigInt type](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

3.1: Integers

Integers

Recall that mathematically, an *integer* is a counting number (1,2,3, ...) along with 0 and the negative counting numbers (-1, -2, -3, ...). Mathematically thinking, there is no largest (or smallest) integer, however, in reality if we are storing a number on a computer (which is a finite device), there must be a limit on the smallest and largest integers to be stored. Practically speaking, we will limit an integer to some number of bits and the standard sizes are 8, 16, 32, 64 and 128.

Unsigned Integers

First, we will examine unsigned integers and typically these are thought of as the non- negative integers (0,1,2,3, ...). For example, 8-bit unsigned integers have a total of $2^8 = 256$ nonnegative numbers and since the smallest is 0, the largest is 255.

In julia, the data types for unsigned integers are `UInt8`, `UInt16`, `UInt32`, `UInt64` and `UInt128` . We can get the smallest and largest value for these with the `typemin` and `typemax` functions.



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
typemin(UInt8)
```

```
0x00
```



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
typemax(UInt8)
```

```
0xff
```

Appendix XXXX covers many of the details of representation integers in binary and performing basic operations. We will cover the a superficial level of integer representation and operations here in this chapter, but for those with desire for more depth see Appendix XXX

In julia, we can use the `bitstring` function to give the binary representation of integers and floating points. For example



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
bitstring(UInt8(18))
```

```
"00010010"
```

and



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
bitstring(UInt8(255))
```

```
"11111111"
```

Similarly, the unsigned integers with more bits work the same with largest range of integers. For example



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
bitstring(UInt64(100000))
```

[illegible]

which is a string of length 64.

Signed Integers

In Julia, the signed integers are `Int8`, `Int16`, `Int32`, `Int64` and `Int128`. Also, there is a integer type `Int` which defaults to the sized integer of the typical integer size on your machine. This is generally `Int64`.

Let's look in detail about 8-bit signed integers. The largest and smallest values that can be stored with Int8 can be found with



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
typemin(Int8)
```

-128



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
typemax(Int8)
```

127

Basically the number between 0 and 127 are identical between Int8 and UInt8.

Overflow and Underflow of integer operations

Again, unlike mathematical integers, any computer-based integer has a maximum and minimum values. In short, if an operation results in a number above the maximum, then there is an overflow error and less than the minimum there is an underflow error.

Here's a simple example with 8-bit integers. Let



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=Int8(95)
```

95

and



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
y=Int8(70)
```

70

The sum of 95 and 70 is 165 and above the maximum value for `Int8`. However, entering





Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x+y
```

```
-91
```

returns a strange result. Perhaps we expected an error. What just happened? If you want to know why the value of -91 arose, dig into the details in Chapter XXX, but the reason why there was no overflow error is that julia does not automatically check for such errors, due to the fact that there is overhead in checking, which will slow down operations.

If you want to check, there are a suite of operations that will check, therefore:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
Base.checked_add(x, y)
```

```
OverflowError: 95 + 70 overflowed for type Int8
```

```
Stacktrace:
```

```
[1] throw_overflowerr_binaryop(::Symbol, ::Int8, ::Int8) at ./checked.jl:154
[2] checked_add(::Int8, ::Int8) at ./checked.jl:166
[3] top-level scope at In[8]:1
[4] include_string(::Function, ::Module, ::String, ::String) at ./loading.jl:1091
```

Go to julia's documentation on `checked_add` which starts a list of functions that will check for over and underflow. If there is any chance of overflow/underflow errors, then the results may be wrong. Keep this in mind as in Chapter XXXX we will write tests for code.

This page titled [3.1: Integers](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

3.2: Floating Point Numbers

Many fields in scientific computing rely on using decimals and the standard way to store these in a computer is with *floating point numbers*. Details on floating-point numbers are in Appendix XXXX. Julia has 16-,32- and 64-bit floating point numbers called `Float16` , `Float32` and `Float64` and by default on most systems is the `Float64` .

There are two limitations to any floating-point number. First, the number of digits stored in the number and secondly, the maximum and minimum values. Each built-in type splits the number of bits into storing both and there is a balance between these. A rule of thumb is that

- `Float16` stores 4 decimal digits and the max is about 32,000.
- `Float32` stores 8 decimal digits and the max is about 10^{38} .
- `Float64` stores 16 decimal digits and the max is about 10^{307}

We can using the `bitstring` function in julia to find the binary representation. Notice that



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
bitstring(Float16(8.625))
```

```
"0100100001010000"
```

Again, details are in Appendix XXXXX but, in short, a floating- point number is stored in scientific notation with the abscissa, exponent and the sign all combined together.

Unlike integers, most numbers cannot be stored exactly with a floating-point number. For example, $1/3$ divides 1 by 3 and results in a floating-point number close to the fraction $\frac{1}{3}$. In julia this is 0.3333333333333333 and also note that



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login


```
bitstring(1/3)
```

```
"0011111110101010101010101010101010101010101010101010101010101"
```

Notice that there are non-zero bits throughout the number in this case that didn't occur with 8.625. This is because as a fraction 8.625 has a denominator of 8, which is a power of 2. If a fraction can be written with such a denominator, the number in binary has 0s that pad the right end of the number.

What does this matter? Consider the following:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

[illegible]

1.0000000000000000000002

is not 1, the expected result. This is an example of the limitations of floating-point numbers and 1) either we deal with it or 2) use a different data type (in this case either a `BigFloat` or `Rational` would be better).

Note: This occurred because the closest floating point to the fraction $1/9$ was just slightly above $1/9$ and adding up 9 of those numbers results in the extra amount

Unless you know you have some reason to choose otherwise, choose `Float64` for most floating-point numbers. There are still underflow and overflow errors associated with it, but as we will see in Chapter XXXXX, generally round-off error associated with floating-point number is more detrimental to calculations.

This page titled [3.2: Floating Point Numbers](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

3.4: Rational and Complex Numbers



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
2//3
-4//7
178//11
8//18
```

One advantage that they have is that the numerator and denominator are stored as integers (64-bit by default) and are not subject to round-off errors that floating points are. The standard operations $+$, $-$, $\cdot$, $\div$ between rationals results in a rational and as we will see in this course, there are advantages to using rationals instead of floating points.

Exercise

- $\frac{1}{2} + \frac{2}{3}$
- $\frac{1}{2} - \frac{2}{3}$
- $\frac{2}{3} \cdot \frac{3}{5}$
- $\frac{2}{3} \div \frac{3}{5}$



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
# insert your code here.
```

The Rational Type

If you enter



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
typeof(1//2)
```

is called a Parametric Composite Type, which will be talked about later. In this particular case, this is a rational type, but inside it (the numerator and denominator), they are type Int64.

For example, to make a different type of rational you need to declare a different integer type inside, enter



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
Int16(1)//Int16(3)
```

3.4: Rational and Complex Numbers is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

CHAPTER OVERVIEW

4: Introduction to Functions

In any language, the function is one of the most important ideas and this chapter covers the introduction of functions with examples in Julia. There are three main purposes of functions in scientific computation.

1. A function in a computer language mimics that of a mathematical function, which are crucial in this field.
2. Functions simplify code. Wherever code is repeated either exactly or nearly exactly, functions are one way to reduce the amount of code written and makes code conceptually easier.
3. Functions abstract code. If you can create a piece of code that does a particular task, then this helps in the abstraction process.
4. Functions allow separation of code. If there is a large code and a section of it does a specific task, making it a function will separate code.

The Julia documentation on functions has additional information on functions and we will cover some advanced features of functions in Chapters XX and XXX

- [4.1: Simple Examples of Functions](#)
- [4.2: Input and Output of Functions](#)
- [4.3: Multiple Dispatch](#)
- [4.4: Variable Number of arguments](#)
- [4.5: Factorial Function](#)
- [4.6: Variable Scope](#)

This page titled [4: Introduction to Functions](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

4.1: Simple Examples of Functions

A simple example of a julia function is



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
function sq(x)
    x^2
end
```

which just returns the square of the argument, called x . The name of the function is `sq` and the `x` inside the parentheses is called the *argument* of the function. Both the function name and any argument must adhere to the rules of variables from Chapter 3.

An alternative way to do this is the following:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
sq(x)=x^2
```

Calling a Function

To use a function or often denoted calling a function is to use the function with known values for the arguments. For example:





Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
sq(3)
```

Exercise

Find the value of the square function for values, 4, -1, 10, 2.5



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
# insert your code here
```

This page titled [4.1: Simple Examples of Functions](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

4.2: Input and Output of Functions

Function Arguments

The function arguments are the numbers or strings or an element of any data type that is passed into the function. In the function `sq` above, the number `x` is the only argument. We can have more arguments by separating by commands. The (quite unnecessary function) `theSum` will take two arguments and add the result:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
theSum(x,y) = x+y
```

```
theSum (generic function with 1 method)
```

Typically, if a function can be written in one line, we will use this style of functions. For more complex functions, use the function keyword as a block of code.

Exercise

Write a function `theMean` that finds the mean (average) of two arguments `x` and `y`.



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
# insert your code here
```

Returning values from a function

The functions `sq` or `theSum` returned the value that is the last line of the function. If you want to return a value before the last line you can use the `return` command. The following will return true if the number is odd and false if the number is true:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
function isOdd(n)
  if mod(n,2)==1
    return true
  end
  false
end
```

isOdd (generic function with 1 method)

The `mod` function is the remainder of `n` divided by 2 and can also be written `n % 2`. If the remainder is 1, then the number is odd. On line #3, return true the code stops here and exits the function and doesn't execute any of the other lines. Note: the `==` tests for equality. This will be discussed in chapter XXX.

A couple of short tests follow:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
isOdd(3)
```

```
true
```




Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
isOdd(10)
```

```
false
```

A shorter way to write this (but doesn't use the return statement) just evaluates if $\text{mod}(n, 2)$ is 1 or not:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
isOdd(n) = mod(n,2)==1
```

```
isOdd (generic function with 1 method)
```

which will return true if $\text{mod}(n, 2)$ is actually 1 and false if it is anything else (but the only other possibility is 0). Again, since this is just one line, we'll use the shorthand notation.

Note: The second `isOdd` function replaces the original one. This is much like the idea of a variable value being updated.

Indentation in Functions

You should notice that the `isOdd` function written above as a block of 5 lines has different indentation. In Julia, the spaces don't matter but it is standard to indent for clarity. Notice first that in all of the functions so far, the code has been indented two spaces and then the if block is indented again 2 spaces. There is nothing special about 2 spaces. This is often a personal style and others use 3 or 4 spaces.

This page titled [4.2: Input and Output of Functions](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

4.3: Multiple Dispatch

Multiple Dispatch

Before starting this, make sure that you have completed the exercise above to write a 2-argument version of the mean. We can update this will argument types as:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
theMean(x::Number,y::Number) = (x+y)/2
```

```
theMean (generic function with 1 method)
```

It would be nice to have a mean function that takes more than 2 numbers as well, so the following is a three-argument version of the mean function can be written



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
theMean(x::Real,y::Real,z::Real) = (x+y+z)/3
```

```
theMean (generic function with 2 methods)
```

Depending on the number of arguments, julia will call the appropriate function. This is an example of Multiple Dispatch, in which either the number or type of arguments determine the actual function call.

If you look above when you declared the functions, you should see that the second one entered said `theMean (generic function with 2 methods)`, which says that there are two functions called mean. Another helpful command is



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
methods(theMean)
```

2 methods for generic function **theMean**:

- `theMean(x::Real, y::Real, z::Real)` in Main at In[3]:1
- `theMean(x::Number, y::Number)` in Main at In[2]:1

and this shows that 1) there are two versions of the function and 2) where they are defined. Note: if you have the functions in a Jupyter notebook then the In[1] and In[2] will correspond to output cells. If instead they are defined in a specific file, the name/location of the file and the line number will be shown.

Multiple dispatch also allows different types of arguments as well. Let's say we want to create a function mean that take a single string as a argument like:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
theMean(str::String) = string("This should return the definition of ",str)
```

```
theMean (generic function with 3 methods)
```

and note that this says there are now 3 method (function definitions).

Exercise

Call the function `theMean` using the following inputs:

- 2 and 3 (Note: separate by commas)
- 2,3,4
- "word"



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
# insert your code here
```

Rerun inside the code block the different arguments and you should notice that the different functions are called.

This page titled [4.3: Multiple Dispatch](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

4.4: Variable Number of arguments

From the last exercise, it would be unfortunate if we have to write different functions for different number of arguments. We can write a variable number of arguments with a ... trailing the last argument. The following is a generalized version of the mean:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
function theMean(x::Number...)
    local sum=0
    for val in x
        sum += val
    end
    sum/length(x)
end
```

which uses a for loop that we will discuss later. This function will now find the mean using any number of arguments.



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
theMean(1, 2, 3, 4)
```

Also, note that the argument `x` with the ... is a tuple, (see section 3.9). An alternative way to access the individual elements of `x` would be to use brackets. For example, `x[3]` would be the third argument.

Exercise

Try the following (replace the code in the above code block) and notice the resulting type:

- `theMean(11//2,5//6,1//9)`
- `theMean(1.0,2.0,3.0,4.0,5.0)`

You should see that each of these return a floating point number. This is because division `/` with any other number returns a floating point.

Multiple Return Values

A very nice feature of Julia functions is that of multiple return values. Instead of only being able to return a single number (or requiring to send an array or structural type), you can return more than 1 number (or other data type). For example.



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
h(x,y) = x+y,x-y
```

and we call this in the same manner. For example,



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
h(3,5)
```

We can store the results of any function that returns multiple arguments in the following example:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
p, q = h(3, 5)
```

the p will take on the value 8 and q will take on the value of -2.

The result of this function is a tuple as we saw in section XXXX. Although in that section we used parentheses around the tuple, it is not necessary and generally isn't used to return a tuple in a function. We will use this for the result of the quadratic formula in Chapter XXXX.

4.4: Variable Number of arguments is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

4.5: Factorial Function

Mathematically, we define the factorial as a function on a non-negative integer as

$$n! = n(n-1)(n-2)\cdots 3 \cdot 2 \cdot 1 \quad (4.5.1)$$

or the product of all of the numbers from itself down to 1. There are a number of ways to program this function as we will see. One way is

```
function fact(n::Integer)
    local prod=1
    for i=1:n
        prod *= i
    end
    prod
end
```

run restart restart & run all

uses a `for` loop and we will see the details of this in Chapter XXX. The `for` loop first assigns `i` the value 1 then executes the lines, then sets the value to 2, then executes the block, and so on until `i` is `n`. Since `prod` starts as 1, this multiplies `prod` by every integer between 1 and `n`, and thus is the factorial. The result in `prod` is returned.

For example, we can call the factorial function with an example like

```
factorial(10)
```

run restart restart & run all

Exercise

Try calling the factorial with a few other integers. Include 0 and negative numbers as well.

4.5: Factorial Function is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

4.6: Variable Scope

If you are using the command line julia (also called the REPL) or iJulia, then entering `x=6` will create `x` as a global variable, even without saying so explicitly. If we consider the factorial function above:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
function fact(n::Integer)
    prod=1
    for i=1:n
        prod *= i
    end
    prod
end
```

then recall that `n` is the function argument and the variable `prod` is actually declared locally implicitly and it is good form to say it is local by



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
function fact(n::Integer)
    local prod=1
    for i=1:n
        prod *= i
    end
```

```
prod  
end
```



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
2*prod
```

Local and Global Variables



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=3  
function f()  
    local x=2  
    x  
end  
f()
```





Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

x



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=3
function g()
  global x=2
  x
end
g()
```



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

x

4.6: Variable Scope is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

CHAPTER OVERVIEW

5: Boolean Statements, Loops and Branching

[5.1: Boolean values and if statements](#)

[5.2: If Statements](#)

[5.3: While Loops](#)

This page titled [5: Boolean Statements, Loops and Branching](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

5.1: Boolean values and if statements

A boolean value is something that is either true or false. These are built-in constants in Julia. Sometimes we will want to know if a statement is true or false, but generally, we will use them in other structures.

We often use boolean to test various conditions. For each, testing equality, or comparison of numbers we use `==`, `<`, `>`, `<=`, `>=` for equality, less than, greater than, less than or equal or greater than or equal respectively.

If we set



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=3
```

and then we will check a few boolean statements containing `x`. For example,



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x==3
```

```
true
```

Check also `x<3`, `x>=3` and `x^2<10` to test a variety of comparisons. Edit the code block above to check.

Compounds boolean statements

We often want to test multiple boolean statements and can build up compound ones with either the “and” (`&&`) or “or” (`||`) operators. Recall the following table for `&&` and `||`

AND	T	F		OR	T	F
T	T	F		T	T	T
F	F	F		F	T	F

If we set



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=3
y=10
```

if we want to test that x is greater than or equal to 0 *and* y is 5, we can enter:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x>=0 && y==5
```

This returns `false` , since only the first is `true` and both must be `true` for this compound statement to be `true` . However,





Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x>=0 || y==5
```

returns `true` since only one of the two need to be `true` .

In both of these examples, it is important to note the order of operations or operator precedence. This was mentioned in section XXX, however as the number of operators grows, it's important to know the precedence. In these cases the tests `==`, `<=`, `<`, `>=`, `>` have precedence over `&&` and `||` . Therefore in the example above, `x>=0` and `y==5` is evaluated first before the `||` .

Also, `&&` has precedence over `||` in that if we evaluate



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x >=0 && y > 7 || y == 5
```

results in `true` . You can think of this resulting in `true && true || false` and because of precedence the first pair is tested (to be true) then the result `true || false` results in `true` .

Often when precedence is unclear, adding parentheses can be helpful. Instead, perhaps write the above as:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login


```
(x >= 0 && y > 7) || y == 5
```

Exercise

If ?? check if ????

This page titled [5.1: Boolean values and if statements](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

5.2: If Statements

An if statement is used to do different things depending on the value of a variable. A standard example of this is the piecewise version of the absolute value. Mathematically, we write:

$$|x| = \begin{cases} x & x \geq 0, \\ -x & x < 0 \end{cases} \quad (5.2.1)$$

We could write this as a function as



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
function absValue(x::Number)
  if x >= 0
    return x
  else
    return -1*x
  end
end
```

Just a check, let's find the absolute value of 3:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
absValue(3)
```

And check the absolute of -7 by editing the code block above.

And If-then-else with More than 2 choices

You may need more than 2 choices on an if statement. Recall the quadrants of the xy -plane start at I for the upper right and increase as you traverse counterclockwise. A function could be written:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
function quadrant(x::Real,y::Real)
  if x > 0 && y > 0
    return "I"
  elseif x < 0 && y > 0
    return "II"
  elseif x < 0 && y < 0
    return "III"
  elseif x > 0 && y < 0
    return "IV"
  else
    return "NONE"
  end
end
```

Notice that the format is that you start with an `if` boolean and then the other checks are `elseif` and then the final one (if nothing is satisfied) is just `else`. Also, the reason we have final else statement is that technically if you are on an axis, then you are not in a quadrant, so that is the reason for the last option.

Ternary If-Then-Else

Often, an if-else statement is quite short and you want to a value depending on a condition. The absolute value example was such an example. There is what is called a ternary if-else statement that has the form:

```
1 | condition ? value_if_condition_is_true : value_if_condition_is_false
```

which returns `value_if_condition_is_true` if condition is true otherwise `value_if_condition_is_false` is returned. The absolute value example above can be written as a single line:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
absVal2(x::Real)=(x>=0) ? x : -x
```

and once you practice with this, it will be easy to read and much shorter (1 line versus 7). Be careful with the syntax of this. It is required that the expressions around the ? be padded with spaces to parse correctly. The error is reasonably clear if you don't write it correctly.

Alternatively, you can write this as



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
absVal3(x::Real)=ifelse(x>=0, x, -1*x)
```

which is the same as the ternary if-then-else with a different syntax.

Exercise

Write the recursive factorial function from Section XXXX using the ternary if-then-else.

This page titled [5.2: If Statements](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

5.3: While Loops

Another very common construction for programming is called a while loop. Basically, we want to run a few statements while some boolean statement is true. Here's a simple, but uninteresting example:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
let
  local n=1
  while n<5
    println(n)
    n+=1
  end
end
```

and note that the expression `n+=1` is shorthand for `n=n+1`. This is a let block, which just encapsulates the content and separates it from global values. It can be thought of like a function except it is just run as is, not declared and then run. Also there are no arguments.

You can see from the output that the numbers 1 to 4 are printed. What happens is that

1. The local variable `n` is set to 1
2. The while loop checks if `n<5`. Since this is `true`, the block is run
3. The value of `n` is printed out as 1 and then updated with the statement `n+=1` from 1 to 2.
4. At the end, we return to the top of the while loop and the statement is checked. Since `2<5`, the while loop block is run.
5. The value of `n` is printed out as 2 and then updated with the statement `n+=1` from 2 to 3.
6. At the end, we return to the top of the while loop and the statement is checked. Since `3<5`, the while loop block is run.
7. The value of `n` is printed out as 3 and then updated with the statement `n+=1` from 3 to 4.
8. At the end, we return to the top of the while loop and the statement is checked. Since `4<5`, the while loop block is run.
9. The value of `n` is printed out as 4 and then updated with the statement `n+=1` from 4 to 5.
10. At the end, we return to the top of the while loop and the statement is checked. Since `5<5` returns false, the while loop block is not run.
11. the `let` block ends and the code block is finished.

Bisection Method

A more practical example of a `while` loop will be the Bisection Method for finding a root



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
function bisection(f::Function,a::Real,b::Real)
  local c
  while (b-a)>1e-6
    c = 0.5*(a+b) # find the midpoint
    # test if f(a) and f(c) have opposite signs
    # that will determine the new interval
    if f(a)*f(c) < 0
      b=c
    else
      a=c
    end
  end # the while loop
  c
end
```

In short, this method takes a function f and an interval $[a, b]$ and continually bisects it ensuring there is a root in the resulting interval. It continues while the length of the interval is greater than $1e-6$ or 10^{-6} . To test it, consider



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

$f(x) = x^2 - 2$

which has a root of 2. The function call



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
bisection(f,1,2)
```

and this result is approximately $\sqrt{2}$.

Note that the if-statement in the bisection function did not use the ternary if-then-else mainly due to the fact that different variables were assigned in each branch. If a single variable would take on different value, then it would probably be clearer to use the ternary if-then-else.

Infinite Loops

It is common in a while loop to keep running it forever. This occurs if there is some bug or you haven't considered all cases. For example, in the bisection method above, if the function doesn't have a root (like $f(x) = x^2 + 2$), then this will never stop.

Here's a few things that can help prevent or debug the code:

- Make sure something is changing in your loop. If you intend to stop the loop on an index, make sure the index is updating.
- Look at your code and see if you have something that you think will stop the loop. What ever is in the boolean statement needs to eventually switch.
- Consider an additional stopping condition. You may need to add a variable to count the number of times you've gone through the loop and stop if it hits some maximum, which is greater than what you would expect.
- Stop the code if you need to. You may need to interrupt the kernel. In the REPL, CTRL-C will stop and in Jupyter, selecting the *Kernel* menu then *Interrupt* should stop it. The square in the toolbar should work too. See section A.7 for more information.

This page titled [5.3: While Loops](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

CHAPTER OVERVIEW

6: Arrays

[5.1: Introduction to Linear Algebra](#)

[5.2: Advanced Topics in Linear Algebra](#)

This page titled [6: Arrays](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

5.1: Introduction to Linear Algebra

This page titled [5.1: Introduction to Linear Algebra](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

5.2: Advanced Topics in Linear Algebra

This page titled [5.2: Advanced Topics in Linear Algebra](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

CHAPTER OVERVIEW

7: Functional Programming and an Introduction to Writing fast code

[6.1: Numerical Integration](#)

[6.2: Differential Equations](#)

[6.3: Optimization](#)

[6.4: Complex Numbers and Analysis](#)

This page titled [7: Functional Programming and an Introduction to Writing fast code](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

6.1: Numerical Integration

This page titled [6.1: Numerical Integration](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

6.2: Differential Equations

This page titled [6.2: Differential Equations](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

6.3: Optimization

This page titled [6.3: Optimization](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

6.4: Complex Numbers and Analysis

This page titled [6.4: Complex Numbers and Analysis](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

CHAPTER OVERVIEW

8: Number theory and Algorithm Development

- [7.1: Calculating pi](#)
- [7.2: Newton's Method in the Complex Plane](#)
- [7.3: Installing and Getting Started with Julia](#)
- [7.4: Loading Packages](#)
- [7.5: Binary Representation of Numbers](#)
- [7.6: References](#)

This page titled [8: Number theory and Algorithm Development](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

7.1: Calculating pi

This page titled [7.1: Calculating pi](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

7.2: Newton's Method in the Complex Plane

This page titled [7.2: Newton's Method in the Complex Plane](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

7.3: Installing and Getting Started with Julia

This page titled [7.3: Installing and Getting Started with Julia](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

7.4: Loading Packages

This page titled [7.4: Loading Packages](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

7.5: Binary Representation of Numbers

This page titled [7.5: Binary Representation of Numbers](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

7.6: References

This page titled [7.6: References](#) is shared under a [CC BY-NC-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

CHAPTER OVERVIEW

9: Algorithm Analysis

9: Algorithm Analysis is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

CHAPTER OVERVIEW

10: Solving Quadratics and Rootfinding

This page is a draft and is under active development.

[10.1: Absolute and Relative Errors](#)

[10.2: Rounding Errors and the Quadratic Formula](#)

[10.3: Re-examining the Quadratic Formula](#)

[10.4: Newton's Method](#)

This page titled [10: Solving Quadratics and Rootfinding](#) is shared under a [not declared](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

10.1: Absolute and Relative Errors

Consider an algorithm that tries to find the value of x^* . If the algorithm actually returns the value x , then there will be some error. The **absolute error** is

$$|x - x^*| \quad (10.1.1)$$

and the **relative error** is

$$\left| \frac{x - x^*}{x^*} \right|. \quad (10.1.2)$$

Often, the **percent error** is helpful as well, which is just the relative error times 100.

Example

Consider an algorithm that returns $x = 0.0153$ and the actual answer is $x^* = 0.0150$. Find both the absolute and relative errors.

The absolute error is $|0.0153 - 0.015| = 0.0003$ and the relative error is

$$\left| \frac{0.0153 - 0.015}{0.015} \right| = 0.02 \quad (10.1.3)$$

or 2%.

We can do this in Julia as follows. Here is the absolute error:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
xstar = 0.0150;  
x = 0.0153  
abs(x-xstar)
```

```
0.00029999999999999999
```

and the relative error is



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
xstar = 0.0150;  
x = 0.0153  
abs((x-xstar)/xstar)
```

```
0.019999999999999997
```

which is the same results as above.

Exercise

Find the relative, absolute and percent error if $x^* = 130.32$ and $x = 130$.



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
# fill in code here.
```

This page titled [10.1: Absolute and Relative Errors](#) is shared under a [not declared](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

10.2: Rounding Errors and the Quadratic Formula

10.2: Rounding Errors and the Quadratic Formula is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

10.3: Re-examining the Quadratic Formula

10.3: Re-examining the Quadratic Formula is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

10.4: Newton's Method

We'll need the following in this section:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
using Pkg; Pkg.add("Plots");
```



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
using Plots
```

Solving an equation is a very important part of mathematics and other scientific fields. However, finding general solutions to equations is not very easy. Consider a cubic equation like

$$15x^3 - 143x^2 + 226x + 280 = 0 \quad (10.4.1)$$

In the spirit of the quadratic formula, there is a cubic formula. Much of the wikipedia page spends time solving the cubic with all possibilities. In short, it's not very easy. In lieu of using such a formula, a more robust approach is to solve it numerically.

This cubic equation on the left side of 10.4.1 actually factors, but finding those factors is quite difficult to do in general. A plot of the function is given with



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
f(x) = 15x^3-143x^2 +226x+280  
plot([f,x->0], -2.5,10, label=["f(x)" "x-axis"])
```

The three intersection points between the red curve (x -axis) and the blue line (the function, $y = f(x)$) are the three roots.

Newton's method starts with a "guess" at the root and then refines it. Let x_0 be the guess, then

We then look at using Newton's method to solve this. The following returns another value for the root.

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} \quad (10.4.2)$$

When Newton's method works (and often does well), the value of x_1 is closer to the true root than x_0 . We have the function $f(x)$ defined in julia above, and can define the derivative with



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
df(x)=45x^2-286x+226
```

Waiting for kernel...

$$df(x) = 45x^2 - 286x + 226 \quad (10.4.3)$$

We can then do a step of Newton's method with



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x0=0
```

```
x1 = x0 - f(x0)/df(x0)
```

We can then do another step of Newton's method by using x_1 as the input value and



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x2 = x1 - f(x1)/df(x1)
```

and note that it appears that the points are getting closer to the root (see the plot above) which is between 0 and -1. A few more iterations of this should get a very accurate value of the root.

Exercise

Perform two more steps of Newton's method. That is find x_3 and x_4 .



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
# insert your code here.
```

A Newton's Method Function

It should appear natural to try to write a function to perform Newton's method for some number of steps. We will walk through the thought process in developing the function. We start with the template for this function. The things that are needed to find a root using Newton's method is 1) the function 2) the derivative and 3) the initial guess. Therefore we will start with



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
function newton(f::Function,df::Function,x0::Number)

end
```

Here are considerations while building the function:

- You will need to do the two steps above many times so you will need a loop. Since you don't know how many times you will need to run through the loop use a while loop and your condition should be that the two steps x_0 and x_1 are apart from each other.
- Checking if two floating-point numbers are equal are generally not a good idea, because they have to be equal to all bits, so instead we will run the while loop while the difference in the numbers x_0 and x_1 are larger than some default (like 10^{-6}).

Here's more a frame of the function:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login


```
function newton(f::Function, df::Function, x0::Number)
    x1 = x0 - f(x0)/df(x0)
    while abs(x1-x0)>1e-6 # while the two numbers are larger than 10^(-6)
        x0 = x1
        x1 = x0 - f(x0)/df(x0)
    end
    x1
end
```

Waiting for kernel...

Using this we can now call this function as



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
newton(f, df, 0)
```

Just to simplify this, we will define dx as $-f(x_0)/df(x_0)$, which is the distance between successive x values, so we'll use this to determine when to stop the loop:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
function newton(f::Function, df::Function, x0::Real)
    local dx=f(x0)/df(x0)
    while abs(dx)>1e-6
```

```
x0 = x0-dx
dx = f(x0)/df(x0)
end
x0
end
```

And just to ensure that the function still returns the same result,



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
newton(f, df, 0)
```

Using Automatic Differentiation

If you have used Computational Algebra Systems like Maple or Mathematica, you know that computers have the ability to differentiate. Julia does not have this capability built-in, although it has some capability of doing some of the feature set of these programs. There is a system called **automatic differentiation** that will compute the exact derivative to a function at a given point. That is, if you have a function $f(x)$ and a number a , it will give you $f'(a)$. The package is called `ForwardDiff` and you may need to add it and then



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
using Pkg; Pkg.add("ForwardDiff");  
using ForwardDiff
```

For example, if you define:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
g(x) =x^2
```

```
g (generic function with 1 method)
```



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
ForwardDiff.derivative(g,3), ForwardDiff.derivative(g,-1)
```

Newton's Method with Automatic Differentiation

Using the `ForwardDiff` package, we can simplify Newton's method and not require that we need to find the derivative in calling it. The following function



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
function newton(f::Function, x0::Real)
    local dx = f(x0)/ForwardDiff.derivative(f,x0)
    while abs(dx)>1e-6
        x0 = x0 - dx
        dx = f(x0)/ForwardDiff.derivative(f,x0)
    end
    x0
end
```

Waiting for kernel...

This page titled 10.4: Newton's Method is shared under a [not declared](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

CHAPTER OVERVIEW

11: Plotting Data and Functions

[11.1: The Plots Package](#)

[11.2: Presenting Results with Visualization, an overview](#)

[11.3: Other Function Plots](#)

[11.4: Plotting Data](#)

Plotting is crucial to understanding functions and visualizing data. There are many ways to do some plots using various packages. Instead of covering details of multiple packages, we will cover the Plots package, which is overlay over many other plotting packages.

However, as you read through this chapter, don't think of this as a how-to guide for using a package, instead, think of the bigger picture of visualizing functions and data.

[11: Plotting Data and Functions](#) is shared under a [CC BY-SA](#) license and was authored, remixed, and/or curated by LibreTexts.

11.1: The Plots Package

This page is a draft and is under active development.

There is a relatively simple, but powerful plotting package called Plots and don't forget to download it as in Appendix XXX. The full documentation is at the Plots.jl website. Recall that once the package is added, enter



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
using Pkg; Pkg.add("Plots")
```



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
using Plots
```

The Plots package tries to unify the syntax for plotting anything. The basic command for plotting data or functions in 2D is the `plot` command, which attempts to plot any object that can be plotted. The next few examples shows this.

Plotting Functions

For plotting a function, simply call `plot` on the function:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
plot(x->x^2)
```

Note: if you are running this on your own computer, your plot may look a bit different than this one with different fonts. This is mainly due to using a different backend, which is explained below.

If you want to specify the x-range, try:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
plot(x->x^2, -2, 2)
```

```
UndefVarError: plot not defined
```

```
Stacktrace:
```

```
[1] top-level scope at In[1]:1
```

```
[2] include_string(::Function, ::Module, ::String, ::String) at ./loading.jl:1091
```

If we want to plot 2 or more functions on the same axes, pass in an array of functions like:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
plot([x->x^2, x->sin(x)], -2, 2)
```

We will also see below how to change other aspects of the plot including the legend, title, labels on the axes, etc.

Exercise

Plot the functions $f(x) = e^{-x^2}$, $g(x) = \ln(x)$ on the interval $[-3, 3]$



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
# add your code here.
```

This page titled [11.1: The Plots Package](#) is shared under a [CC BY-SA](#) license and was authored, remixed, and/or curated by [Peter Staab](#).

11.2: Presenting Results with Visualization, an overview

11.2.1 Plotting Functions and other Curves

Mathematically speaking, we often think about functions. However, functions can take on many forms including:

- functions of one variable—plots of this are often function plots with the independent variable on the horizontal axis and function values on the vertical. We saw such function graphs above.
- parametric functions (vector functions in 2D). These are functions in which the x and y variables depend on a parameter (often t or θ). We will see how to plot this in section XXX
- implicit curves .An implicit curve is the set of points (x, y) in which $f(x, y) = 0$.The classic example is the circle

$$x^2 + y^2 = 1 \quad (11.2.1)$$

which can be written in the form $f(x, y) = 0$ by subtracting 1 from both sides.

- functions of two variables. These often have the form:

$$z = f(x, y) \quad (11.2.2)$$

and that the two independent variables are x and y and the third variable is the height of the function. There are at least three standard ways of representing such a function:

- surface plots as in section 11.3.3 which is a 3D rendering of the surface
- contour plots (section ??), which generates a curve in the plane for a given number of heights.
- heatmaps (section??) which gives a color representing the height of the function.
- Vector Functions in 3D are often represented as parametric functions of the form:

$$\langle x(t), y(t), z(t) \rangle \quad (11.2.3)$$

where each function gives the x , y or z coordinate at a time t . Examples of this are in section XXX

11.2: Presenting Results with Visualization, an overview is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

11.3: Other Function Plots

Here's some preliminary commands to run if they haven't been yet.



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
using Pkg; Pkg.add("Plots")
```

```
Updating registry at `~/.julia/pkg/registries/General`
Resolving package versions...
Updating `~/Project.toml`
 [91a5bcdd] + Plots v1.14.0
Updating `~/Manifest.toml`
 [6e34b625] + Bzip2_jll v1.0.6+5
 [83423d85] + Cairo_jll v1.16.0+6
 [35d6a980] + ColorSchemes v3.12.1
 [5ae413db] + EarCut_jll v2.1.5+1
 [2e619515] + Expat_jll v2.2.7+6
 [c87230d0] + FFMPEG v0.4.0
 [b22a6f82] + FFMPEG_jll v4.3.1+4
 [a3f928ae] + Fontconfig_jll v2.13.1+14
 [d7e528f0] + FreeType2_jll v2.10.1+5
 [559328eb] + FriBidi_jll v1.0.5+6
 [0656b61e] + GLFW_jll v3.3.4+0
 [28b8d3ca] + GR v0.57.4
 [d2c73de3] + GR_jll v0.57.2+0
 [5c1252a2] + GeometryBasics v0.3.12
 [78b55507] + Gettext_jll v0.20.1+7
 [7746bdde] + Glib_jll v2.59.0+4
 [aacddb02] + JpegTurbo_jll v2.0.1+3
 [c1c5ebd0] + LAME_jll v3.100.0+3
 [dd4b983a] + LZ0_jll v2.10.0+3
 [b964fa9f] + LaTeXStrings v1.2.1
 [23fbe1c1] + Latexify v0.15.5
 [dd192d2f] + LibVPX_jll v1.9.0+1
 [e9f186c6] + Libffi_jll v3.2.1+4
 [d4300ac3] + Libgcrypt_jll v1.8.5+4
```

```
[7e76a0d4] + Libglvnd_jll v1.3.0+3
[7add5ba3] + Libgpg_error_jll v1.36.0+3
[4b2f31a3] + Libmount_jll v2.34.0+3
[89763e89] + Libtiff_jll v4.1.0+2
[38a345b3] + Libuuid_jll v2.34.0+7
[77ba4419] + NaNMath v0.3.5
[e7412a2a] + Ogg_jll v1.3.4+2
[91d4177d] + Opus_jll v1.3.1+3
[2f80f16e] + PCRE_jll v8.42.0+4
[30392449] + Pixman_jll v0.40.0+0
[ccf2f8ad] + PlotThemes v2.0.1
[995b91a9] + PlotUtils v1.0.10
[91a5bcdd] + Plots v1.14.0
[ea2cea3b] + Qt5Base_jll v5.15.2+0
[01d81517] + RecipesPipeline v0.3.2
[f50d1b31] ↑ Rmath_jll v0.2.2+1 ⇒ v0.2.2+2
[6c6a2e73] + Scratch v1.0.3
[09ab397b] + StructArrays v0.5.1
[a2964d1f] + Wayland_jll v1.17.0+4
[2381bf8a] + Wayland_protocols_jll v1.18.0+4
[aed1982a] + XSLT_jll v1.1.33+4
[4f6342f7] + Xorg_libX11_jll v1.6.9+4
[0c0b7dd1] + Xorg_libXau_jll v1.0.9+4
[935fb764] + Xorg_libXcursor_jll v1.2.0+4
[a3789734] + Xorg_libXdmcp_jll v1.1.3+4
[1082639a] + Xorg_libXext_jll v1.3.4+4
[d091e8ba] + Xorg_libXfixes_jll v5.0.3+4
[a51aa0fd] + Xorg_libXi_jll v1.7.10+4
[d1454406] + Xorg_libXinerama_jll v1.1.4+4
[ec84b674] + Xorg_libXrandr_jll v1.5.2+4
[ea2f1a96] + Xorg_libXrender_jll v0.9.10+4
[14d82f49] + Xorg_libpthread_stubs_jll v0.1.0+3
[c7cfdc94] + Xorg_libxcb_jll v1.13.0+3
[cc61e674] + Xorg_libxkbfile_jll v1.1.0+4
[12413925] + Xorg_xcb_util_image_jll v0.4.0+1
[2def613f] + Xorg_xcb_util_jll v0.4.0+1
[975044d2] + Xorg_xcb_util_keysyms_jll v0.4.0+1
[0d47668e] + Xorg_xcb_util_renderutil_jll v0.3.9+1
[c22f9ab0] + Xorg_xcb_util_wm_jll v0.4.1+1
[35661453] + Xorg_xkbcomp_jll v1.4.2+4
[33bec58e] + Xorg_xkeyboard_config_jll v2.27.0+4
[c5fb5394] + Xorg_xtrans_jll v1.4.0+3
[0ac62f75] + libass_jll v0.14.0+4
[f638f0a6] + libfdk_aac_jll v0.1.6+4
[b53b4c65] + libpng_jll v1.6.37+6
[f27f6e37] + libvorbis_jll v1.3.6+6
[1270edf5] + x264_jll v2020.7.14+2
```

```
[dfaa095f] + x265_jll v3.0.0+3  
[d8fb68d0] + xkbcommon_jll v0.9.1+5
```



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

using Plots

Parametric Plots

Recall that a parametric curve is a set of points in the xy -plane given by $(x(t), y(t))$ for functions $x(t)$ and $y(t)$. The variable t is called the parameter. A classic example is the circle that can be written as

$$x(t) = \cos t, \quad y(t) = \sin t$$

To plot the circle using this form, enter



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
plot(t->cos(t), t->sin(t), 0, 2*pi, legend=false)
```

```
UndefVarError: plot not defined
```

```
Stacktrace:
```

```
[1] top-level scope at In[1]:1
```

```
[2] include_string(::Function, ::Module, ::String, ::String) at ./loading.jl:1091
```

and note that the legend is turned off, since with one curve, it doesn't make much sense. Notice that this should be a circle, but it looks like an ellipse due to the aspect ratio. If one instead adds the `aspect_ratio=equal` option, as in



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
plot(t->cos(t),t->sin(t),0,2*pi,aspect_ratio=:equal, legend=false)
```

Exercise

Produce a plot of the curve $x(t) = t^3 - t, y(t) = t^2$ for $-2 \leq t \leq 2$.



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
# insert your code here
```

Implicit Curves

An implicit curve is the set of points such that $f(x, y) = 0$ (or any constant) and a circle is the classic example. For example, $x^2 + y^2 = 1$. Although there are others ways of doing this, we will use some algebra to write the circle as $f(x, y) = x^2 + y^2 - 1$.

We can plot this with the `contour` function for example



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
contour(-1.05:0.05:1.05, -1.05:0.05:1.05, (x,y) -> x^2+y^2-1, levels=[0], aspect_ratio=1)
```

Note that again, we have used the option `aspect_ratio = :equal` to ensure that the circle looks like a circle. The resulting plot is exactly the same as the circle above.

The following example is a bit more visually interesting. This is the function $f(x, y) = \sin(x + y) - \cos(xy) + 1$



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
contour(-10.1:0.1:10.1, -10.1:0.1:10.1, (x,y) -> sin(x+y)-cos(x*y)+1, levels=[0], aspect_ratio=1)
```

Exercise

A cardioid is a 2D curve that looks a bit like a heart. It can be represented by an implicit curve with the equation

$$(x^2 + y^2)^2 + 4ax(x^2 + y^2) - 4a^2y^2 = 0$$

Plot the cardioid with $a = 1$ using the code block below



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
# insert your code here
```

Surface Plots

If we have a function of 2 variables, a surface plot is nice to use. For example, if we have the function

$$f(x, y) = e^{-0.1(x^2+y^2)} \quad (11.3.1)$$

and we want to plot it from -3 to 3 in both directions, if we define



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
f(x,y)=exp(-0.1*(x^2+y^2))  
x = y = range(-5, stop = 5, length = 40)
```

```
-5.0:0.2564102564102564:5.0
```

and then plot with



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
surface(x,y, f, legend = false)
```

```
UndefVarError: surface not defined
```

```
Stacktrace:
```

```
[1] top-level scope at In[2]:1
```

```
[2] include_string(::Function, ::Module, ::String, ::String) at ./loading.jl:1091
```

Exercise

Produce a surface plot of the function $f(x, y) = \sin x \cos y$ for $0 \leq x \leq 2\pi, 0 \leq y \leq 2\pi$ using the code block below



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
# insert your code here
```

Heat Maps

A heat map is a 2-dimensional version of a surface plot in which the height of each value is given a color. The following produces a heat map of the function in [11.3.1](#)



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
heatmap(x, y, f)
```

```
UndefVarError: heatmap not defined
```

```
Stacktrace:
```



```
[1] top-level scope at In[2]:1  
[2] include_string(::Function, ::Module, ::String, ::String) at ./loading.jl:1091
```

Exercise

Produce a surface plot of the function $f(x, y) = \sin x \cos y$ for $0 \leq x \leq 2\pi, 0 \leq y \leq 2\pi$ using the code block below



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
# Insert your code here
```

11.3: Other Function Plots is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

11.4: Plotting Data

Here's some preliminary commands to run if they haven't been yet.



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
using Pkg; Pkg.add("Plots")
```

```
Updating registry at `~/.julia/pkg/registries/General`  
Resolving package versions...  
No Changes to `~/Project.toml`  
No Changes to `~/Manifest.toml`
```

And for this section, we will need to let julia know to use the Plots package:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
using Plots
```

We will want to produce plots of datasets, we'll start with just some random data. The following produces the values 1 to 10 for x and random integers between 1 and 10 for y:



Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
x=1:10  
y=rand(1:10,10)
```

```
10-element Array{Int64,1}:  
 1  
 6  
 3  
 4  
 1  
 3  
 3  
 9  
 8  
 1
```

The following will produce a scatter plot of the data, where each point is plotted as a point.

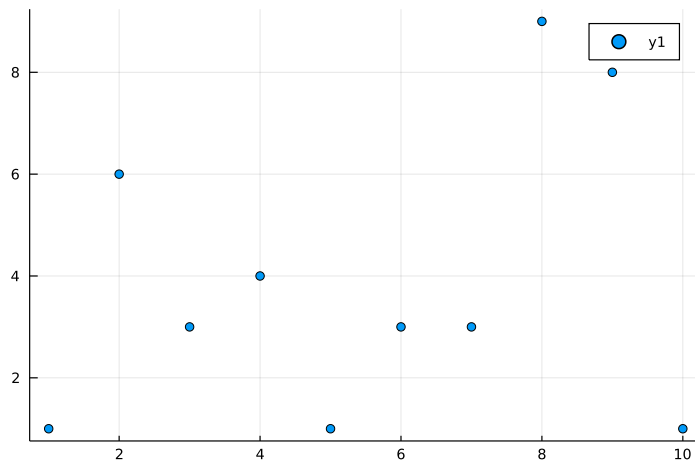


Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
scatter(x,y)
```



If we wanted the data to be plotted with lines connecting the points, we use the same `plot` command as we did above.

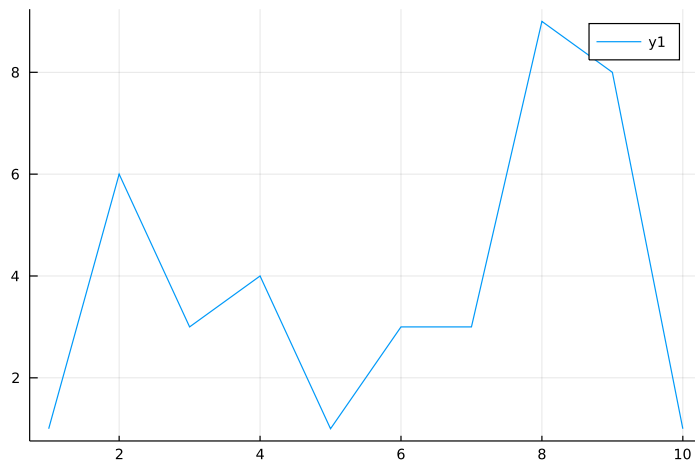


Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
plot(x,y)
```



and we can plot both points and lines with:

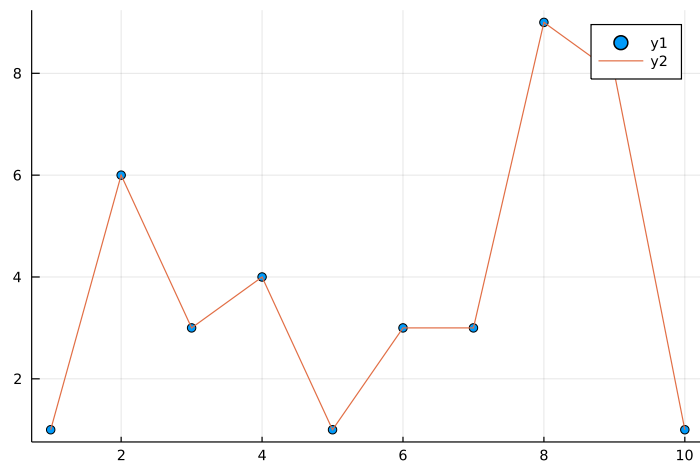


Login with LibreOne to run this code cell interactively.

If you have already signed in, please refresh the page.

Login

```
plot(x,y,seriestype=[:scatter,:line])
```



11.4: Plotting Data is shared under a [not declared](#) license and was authored, remixed, and/or curated by LibreTexts.

Index

D

dire

Glossary

Sample Word 1 | Sample Definition 1

Glossary

Sample Word 1 | Sample Definition 1

Detailed Licensing

Overview

Title: Scientific Computing (Staab)

Webpages: 67

Applicable Restrictions: Noncommercial

All licenses found:

- [CC BY-NC-SA 4.0](#): 73.1% (49 pages)
- [Undeclared](#): 23.9% (16 pages)
- [CC BY-SA 4.0](#): 3% (2 pages)

By Page

- Scientific Computing (Staab) - [CC BY-NC-SA 4.0](#)
 - Front Matter - [CC BY-NC-SA 4.0](#)
 - [TitlePage](#) - [CC BY-NC-SA 4.0](#)
 - [InfoPage](#) - [CC BY-NC-SA 4.0](#)
 - [Table of Contents](#) - [Undeclared](#)
 - [Licensing](#) - [Undeclared](#)
 - [Preface](#) - [CC BY-NC-SA 4.0](#)
 - 1: Introduction to Scientific Computing - [CC BY-NC-SA 4.0](#)
 - [1.1: Examples of Scientific Computing](#) - [CC BY-NC-SA 4.0](#)
 - [1.2: Modeling](#) - [CC BY-NC-SA 4.0](#)
 - [1.3: Computing](#) - [CC BY-NC-SA 4.0](#)
 - [1.4: Ideas needed to do Effective Scientific Computing](#) - [CC BY-NC-SA 4.0](#)
 - [1.5: Examples of Scientific Computing](#) - [CC BY-NC-SA 4.0](#)
 - [1.6: Writing Code for Scientific Computing](#) - [CC BY-NC-SA 4.0](#)
 - [1.7: Let's get started](#) - [CC BY-NC-SA 4.0](#)
 - 2: Storing number and strings and using Julia syntax - [CC BY-NC-SA 4.0](#)
 - [2.1: Numbers](#) - [CC BY-NC-SA 4.0](#)
 - [2.2 Assignment Statement and Variables](#) - [CC BY-NC-SA 4.0](#)
 - [2.3: Strings](#) - [CC BY-NC-SA 4.0](#)
 - [2.4: Expressions and Operator Precedence](#) - [CC BY-NC-SA 4.0](#)
 - 3: Introduction to Data Types - [CC BY-NC-SA 4.0](#)
 - [3.1: Integers](#) - [CC BY-NC-SA 4.0](#)
 - [3.2: Floating Point Numbers](#) - [CC BY-NC-SA 4.0](#)
 - [3.3: Extending integers, the BigInt type](#) - [CC BY-NC-SA 4.0](#)
 - [3.4: Rational and Complex Numbers](#) - [Undeclared](#)
 - 4: Introduction to Functions - [CC BY-NC-SA 4.0](#)
 - [4.1: Simple Examples of Functions](#) - [CC BY-NC-SA 4.0](#)
 - [4.2: Input and Output of Functions](#) - [CC BY-NC-SA 4.0](#)
 - [4.3: Multiple Dispatch](#) - [CC BY-NC-SA 4.0](#)
 - [4.4: Variable Number of arguments](#) - [Undeclared](#)
 - [4.5: Factorial Function](#) - [Undeclared](#)
 - [4.6: Variable Scope](#) - [Undeclared](#)
 - 5: Boolean Statements, Loops and Branching - [CC BY-NC-SA 4.0](#)
 - [5.1: Boolean values and if statements](#) - [CC BY-NC-SA 4.0](#)
 - [5.2: If Statements](#) - [CC BY-NC-SA 4.0](#)
 - [5.3: While Loops](#) - [CC BY-NC-SA 4.0](#)
 - 6: Arrays - [CC BY-NC-SA 4.0](#)
 - [5.1: Introduction to Linear Algebra](#) - [CC BY-NC-SA 4.0](#)
 - [5.2: Advanced Topics in Linear Algebra](#) - [CC BY-NC-SA 4.0](#)
 - 7: Functional Programming and an Introduction to Writing fast code - [CC BY-NC-SA 4.0](#)
 - [6.1: Numerical Integration](#) - [CC BY-NC-SA 4.0](#)
 - [6.2: Differential Equations](#) - [CC BY-NC-SA 4.0](#)
 - [6.3: Optimization](#) - [CC BY-NC-SA 4.0](#)
 - [6.4: Complex Numbers and Analysis](#) - [CC BY-NC-SA 4.0](#)
 - 8: Number theory and Algorithm Development - [CC BY-NC-SA 4.0](#)
 - [7.1: Calculating pi](#) - [CC BY-NC-SA 4.0](#)
 - [7.2: Newton's Method in the Complex Plane](#) - [CC BY-NC-SA 4.0](#)
 - [7.3: Installing and Getting Started with Julia](#) - [CC BY-NC-SA 4.0](#)
 - [7.4: Loading Packages](#) - [CC BY-NC-SA 4.0](#)
 - [7.5: Binary Representation of Numbers](#) - [CC BY-NC-SA 4.0](#)

- 7.6: References - *CC BY-NC-SA 4.0*
- 9: Algorithm Analysis - *Undeclared*
- 10: Solving Quadratics and Rootfinding - *Undeclared*
 - 10.1: Absolute and Relative Errors - *Undeclared*
 - 10.2: Rounding Errors and the Quadratic Formula - *Undeclared*
 - 10.3: Re-examining the Quadratic Formula - *Undeclared*
 - 10.4: Newton's Method - *Undeclared*
- 11: Plotting Data and Functions - *CC BY-SA 4.0*
 - 11.1: The Plots Package - *CC BY-SA 4.0*
 - 11.2: Presenting Results with Visualization, an overview - *Undeclared*
 - 11.3: Other Function Plots - *Undeclared*
 - 11.4: Plotting Data - *Undeclared*
- Back Matter - *CC BY-NC-SA 4.0*
 - Index - *CC BY-NC-SA 4.0*
 - Glossary - *CC BY-NC-SA 4.0*
 - Glossary - *CC BY-NC-SA 4.0*
 - Detailed Licensing - *Undeclared*